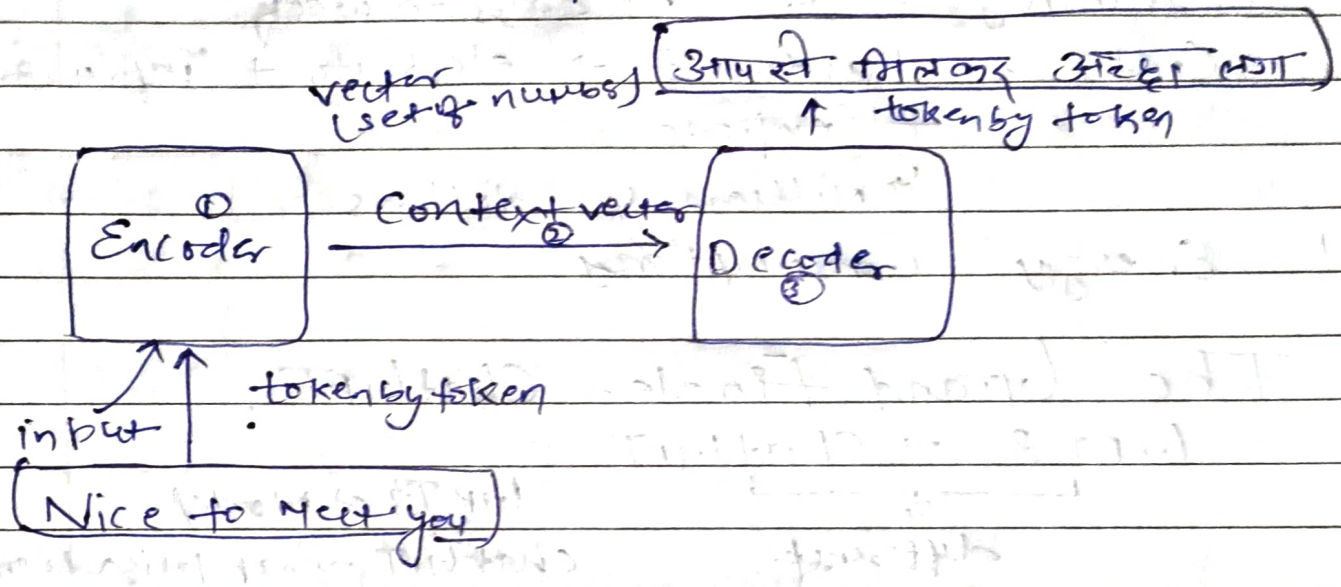
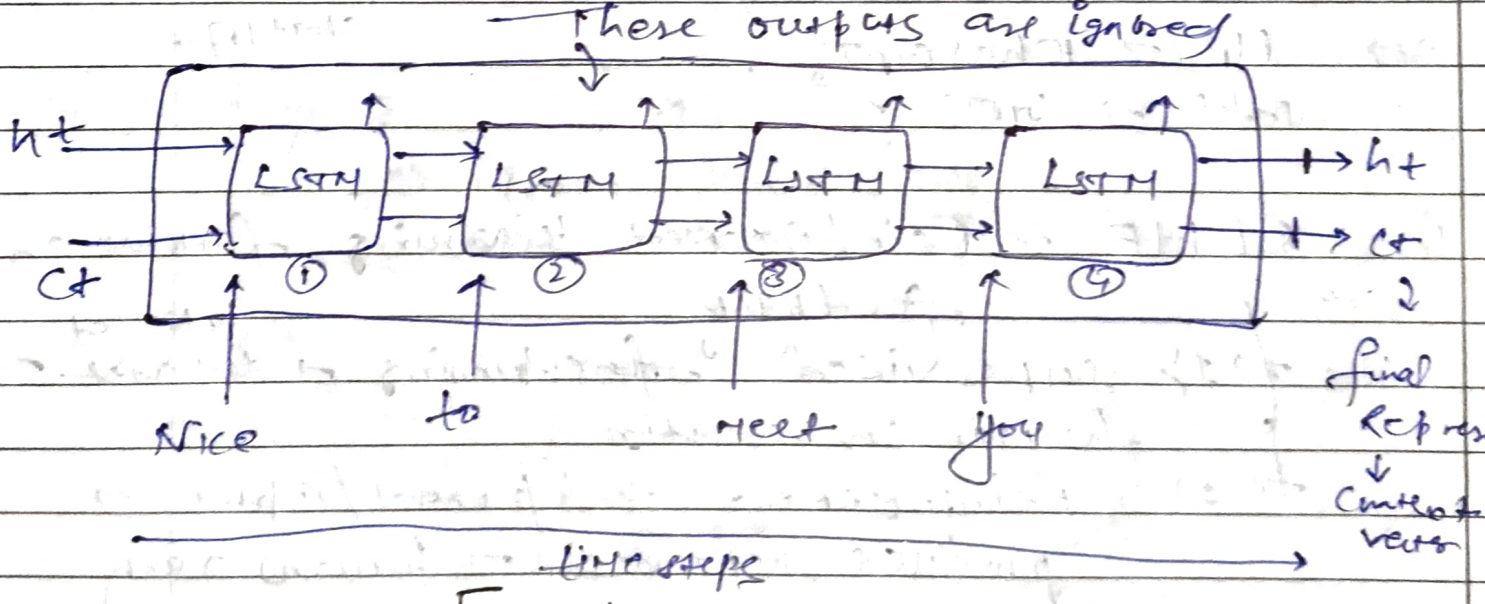
→ Encoder Decoder Architecture / Sequence to sequence "

1) Tabular data → ANN
2) Image based data → CNN
3) Sequential data → RNN / LSTM / GRU
4) Sequence 2 sequence data
   input seq → output seq → Machine translation



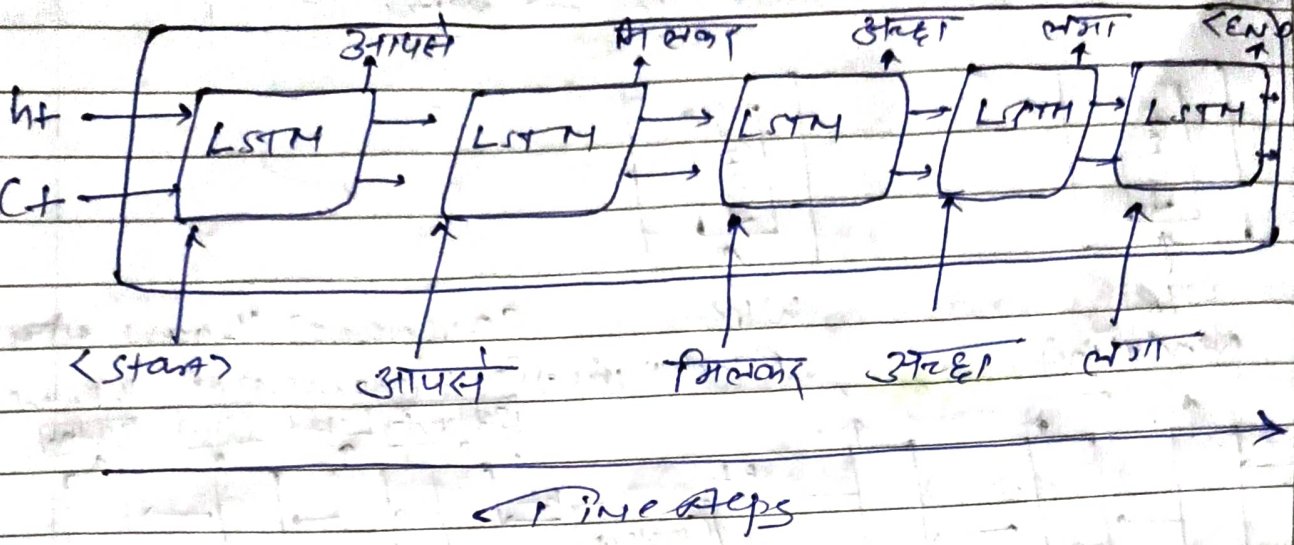


Encoder.

LSTM / GRU ✓ RNN ✗ due to vanishing gradient problem

dataset : kaggle.com/datasets/preetviradiya/english-hindi-dataset

SOS → Decoder



→ Training the Architecture using backpropagation.

→ dataset

eng	Hindi
1) Think about it	सोच लो
2) Come in	अंदर आ जाओ

Step1) tokenization

[think, about, it] [सोच, लो]
[come, in] [अंदर, आ, जाओ]

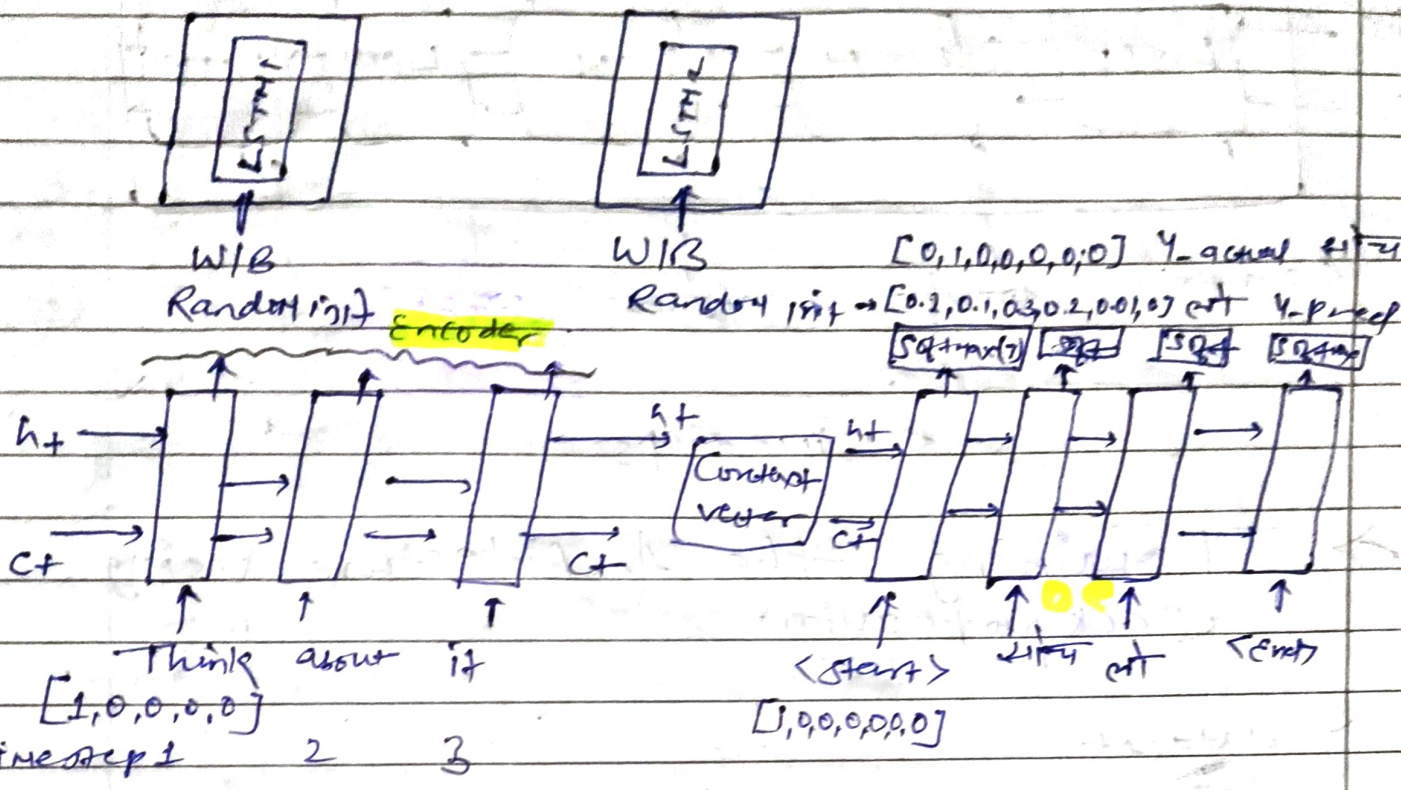
Step2) Convert it into Numbers (OHE)

Eng	Hindi
think [1, 0, 0, 0, 0]	<start> [1, 0, 0, 0, 0, 0, 0] → start
about [0, 1, 0, 0, 0]	<end> [0, 1, 0, 0, 0, 0, 0] → सोच
it [0, 0, 1, 0, 0]	
come [0, 0, 0, 1, 0]	
in [0, 0, 0, 0, 1]	

Step3) start the training.

SdS

→ row 1. → [Think about it → सोचना है]



→ Softmax layer has neurons (Multiclass)
no. of neurons = = no. of unique words in the input / dataset.

→ [Teacher forcing → it is a technique in which if our decoder LSTM (NN) do some error then we ignore that error and send the correct output respective to it. for fast training and high Convergence to output.

→ Step 4) After the prediction (training) we calculate the loss using formula $|Y_{pred} - Y_{actual}|$

Loss → categorical cross entropy used due to Multiclass (7)

SDS

$$L = -\sum_{i=1}^{7} y_i^{\text{true}} \log(y_i^{\text{pred}})$$

Now,

1) forward propogation (train data) ✓
2) loss calculation ✓
3) BACK propogation { → Gradient (calculation)
   { update parameter

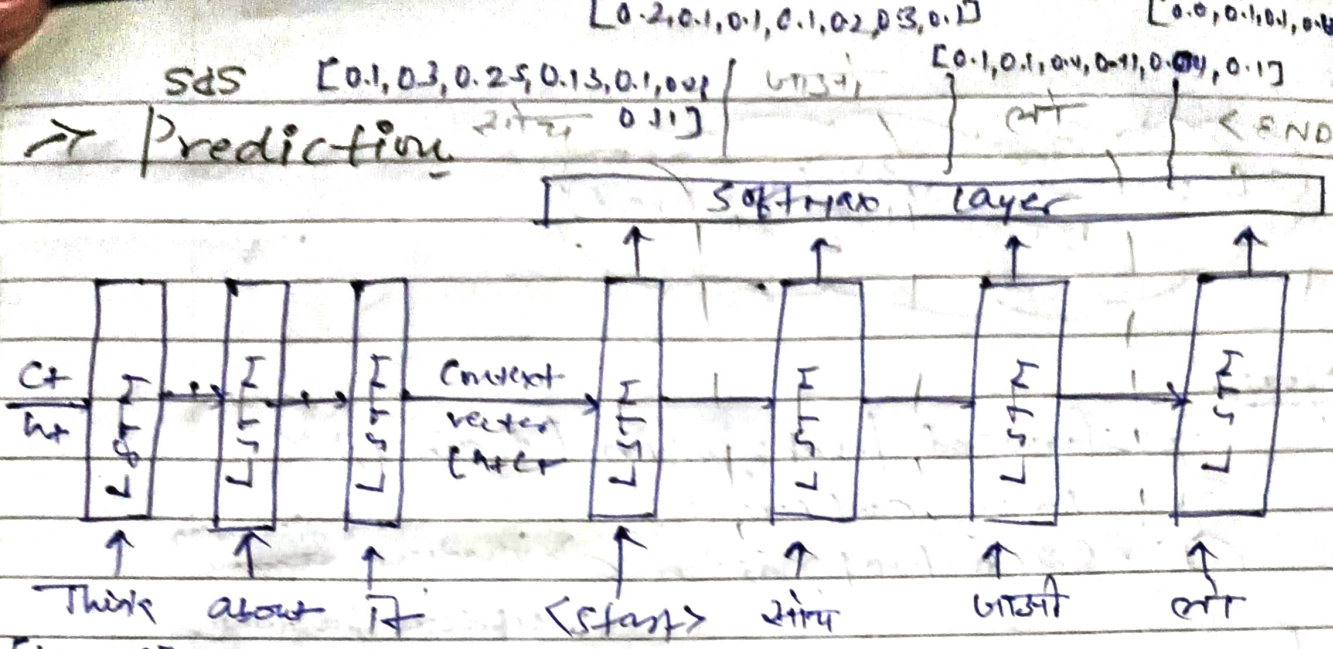
- Gradient represent that what is the contribution (percentage) of a parameter in the loss. and in which direction we adjust that parameter to reduce the loss.

- Update parameter → now after calculating the gradient we update the parameter with the help of gradient descent (multiple type of) we do adjustment with the help of learning rate (η) that tells how much fast it change

flow:

(initial weight) → forward propogation → output → loss
↑ true compare ↓
new weights ← learning rate ← weights updated ← gradient (all parameters (Adam Optimizer) LSTM, SGD, RMS

→ Prediction



[1,0,0,0,0]

i/p → [Think about it]

o/p → सोच जाओ लो

Trained

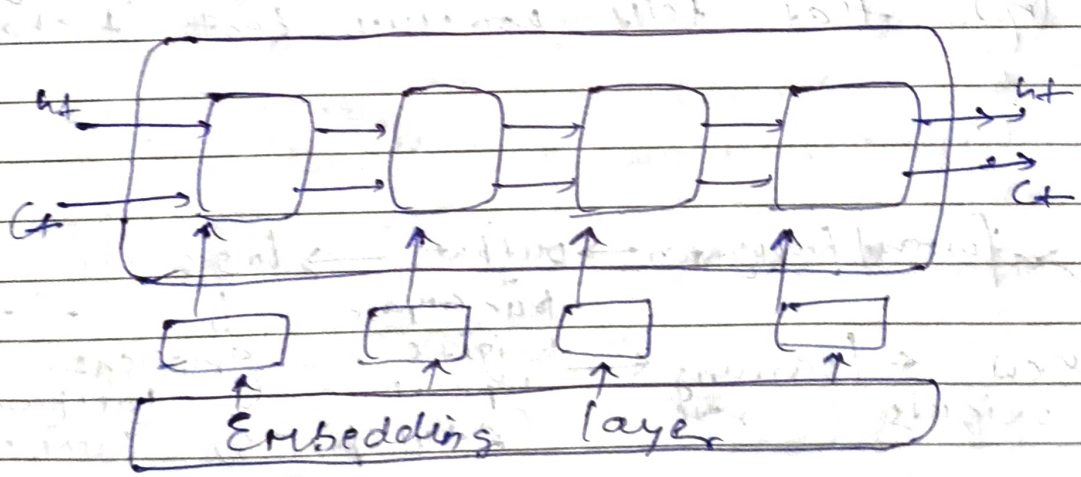
<Start> 1 ; सोच → 2, लो → 3,

अंदर → 4, आ → 5, जाओ → 6,

<End> 7.

→ Improvement-1 [Embedding]

Encoder



- Embeddings are low dimension and dense (less numbers are zero)
- Summary of word ; we can capture context of word.

SdS

types of embedding used → pretrained e.g. word2vec, Glove
→ train with our model
due to that we capture our data ~~notes~~, nuances, subtleties and context generated according to our data

- In Decoder

[embedding layer]
↑
<start>

→ [Improvement 2 [Deep LSTM]
3 layer LSTM

↑ input provided to next LSTM
[LSTM 1] → propagate hidden state

→ why? improvements
1) long term dependency
2) layered representation understanding
3) Deep learning NN parameters ↑↑
learning capability ↑↑
↳ better generalisation
↳ even if variations in our data then give good outputs.

→ Improvement 3 - Reverse the input
it works well only in languages where initial context is high (Eg → french)